MCP

Del cuarto cerrado al mundo real

MexAI / CIATEQ Zapopan / 30 septiembre 2026



Sobre Mi

¡Qué onda! Soy Alex.

Desarrollador de software que terminó metido hasta el cuello en la IA.

Trabajo como **AI Engineer** en el **IIEG** y estudio una **maestría en Ingeniería y Ciencia de Datos**.

Me gusta experimentar con modelos, desarrollar agentes y conectar sistemas que originalmente no estaban pensados para trabajar con IA.

Mi especialidad es tener más proyectos que tiempo libre.



¿Quién sabe qué es MCP?

ACTO I

Un cerebro encerrado

Leyó todo internet.
Le faltó salir del cuarto.

Lo encerramos. Que aprenda todo lo que pueda.

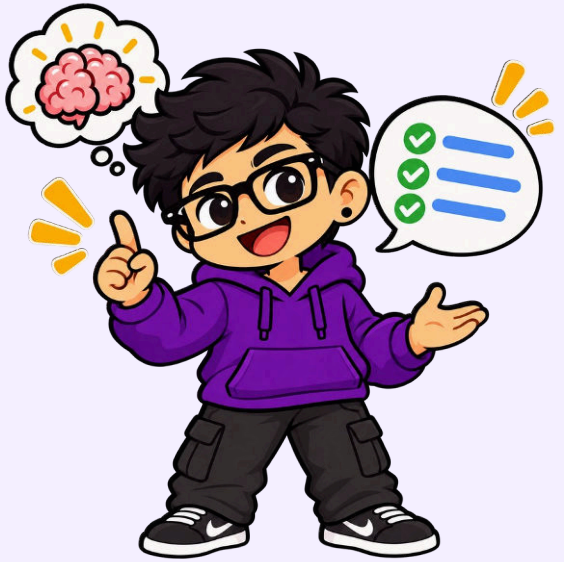
Libros, foros, tutoriales…



Le preguntas.
Se acuerda.
Te responde.

Hasta aquí, todo bien.

“Eso lo leí…
sí me acuerdo.”



PERO A VECES... SALE EL ALUCÍN

¿Cuánto vendimos ayer?

Fuente: créeme, bro.

**“$85,000.
Sin duda, mi estimado.”**



ACTO II

El cerebro que hace trampa

Si no se lo sabe…
le pasamos el acordeón.

Se acabó lo que sabe. ¿Y ahora?

Reentrenar

Actualizar lo aprendido
con más entrenamiento.



“Reentrenar”

Pasarle información
justo cuando la necesita.

Spoiler: el segundo lleva comillas. Es contexto → RAG.

Pasale la informacion por debajo de la mesa



Pregunta → recuperar → contexto → respuesta

SABER EXPLICAR ≠ EJECUTAR

“No me expliques. ¡Hazlo tú!”

Genera la factura. →

1. Abre el sistema…
2. Captura los datos…

¡Pero hazla tú! →

Claro, aquí tienes las instrucciones…

Sin herramientas: más texto. Facturas generadas: 0.

ACTO III

Le damos un cuerpo

Ya da consejos.
Ahora que haga algo.

Del “te explico” al “lo hago”.

LLM

el cerebro



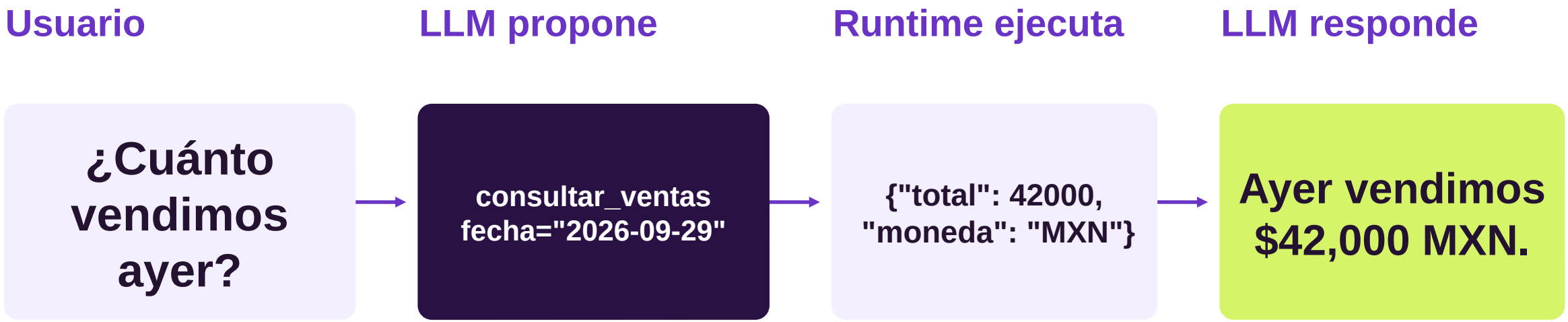
TOOLS

las capacidades

AGENTE = decide + ejecuta + observa

UNA VUELTA DEL CICLO

Una llamada. Un resultado. Una respuesta.



Tool calling ya existía antes de MCP.

¿Y cómo se conecta a mis otras aplicaciónes?





ACTO IV

El infierno de las integraciones

“Solo hay que conectarlo a la API”.

“Solo”

OAuth

refresh tokens

documentación



scopes

permisos

SDK v2 / API v3

Normalmente pronunciado por alguien que no va a conectarlo.

MCP

Un enchufe común.

Descubrir capacidades.
Invocarlas con un contrato compartido.

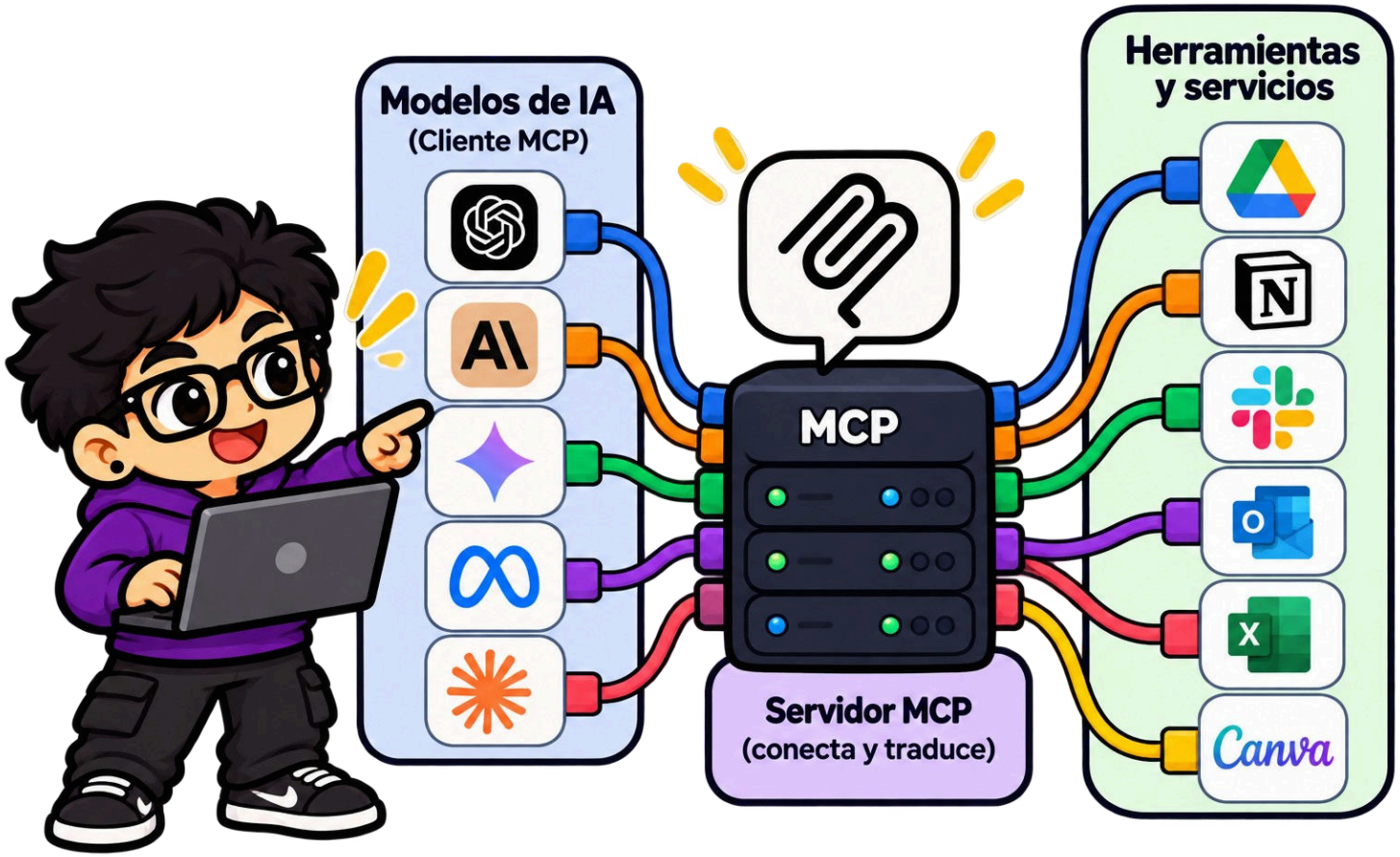
Model Context Protocol



A mano.



Con MCP.


Modelos de IA
(Cliente MCP)
MCP
Servidor MCP
(conecta y traduce)
Herramientas
y servicios
Canva

¿Cómo llego? ¿Dónde? ¿Qué pido?

01

TRANSPORTE

STDIO / HTTP

Cómo viaja el mensaje

02

PUNTO DE ENTRADA

https://api.com/mcp

Dónde hablo · sólo en HTTP

03

MENSAJE

JSON-RPC 2.0

Qué método y argumentos envío

Mismo protocolo. Distinto camino.



STDIO

Proceso local
stdin / stdout · sin URL

STREAMABLE HTTP

Servicio independiente
https://api.com/mcp

EL SERVIDOR OFRECE UN CATÁLOGO

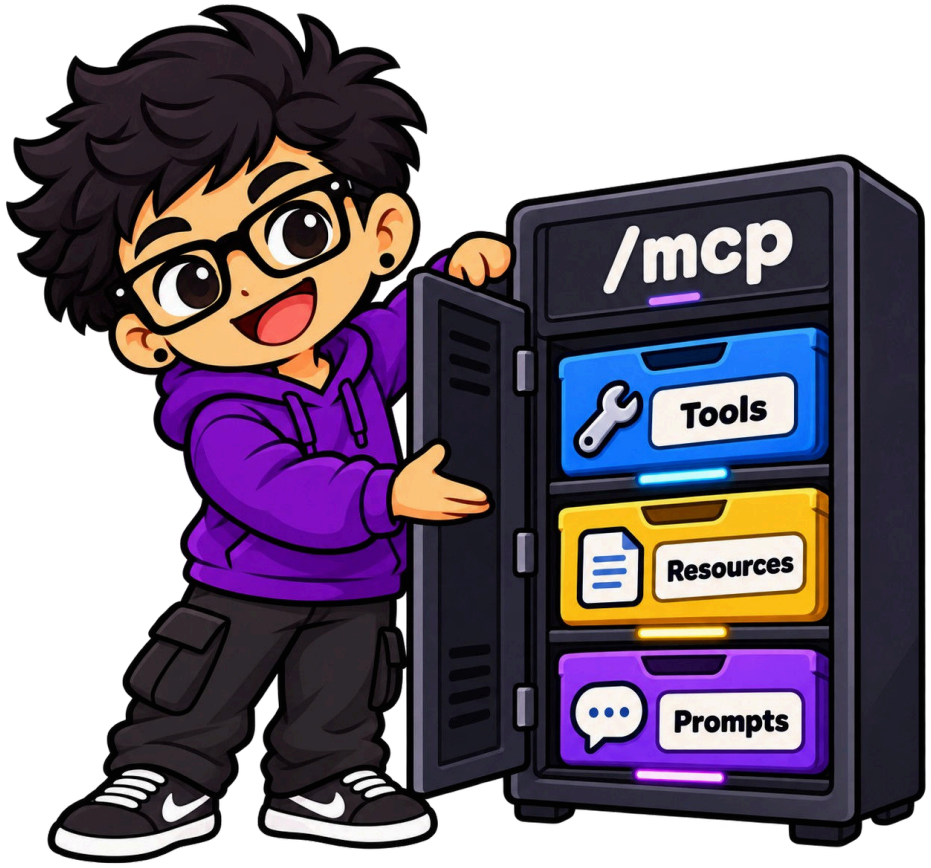
Qué puedes hacer, leer y preparar.

TOOLS	RESOURCES	PROMPTS
`crear_ticket(título)`	`expediente://123`	`resumir_expediente`
Ejecutar	Leer contexto	Usar una plantilla

`Capacidades → tools/list → tools/call → resultado`

CAPACIDADES / MÉTODOS, NO RUTAS REST

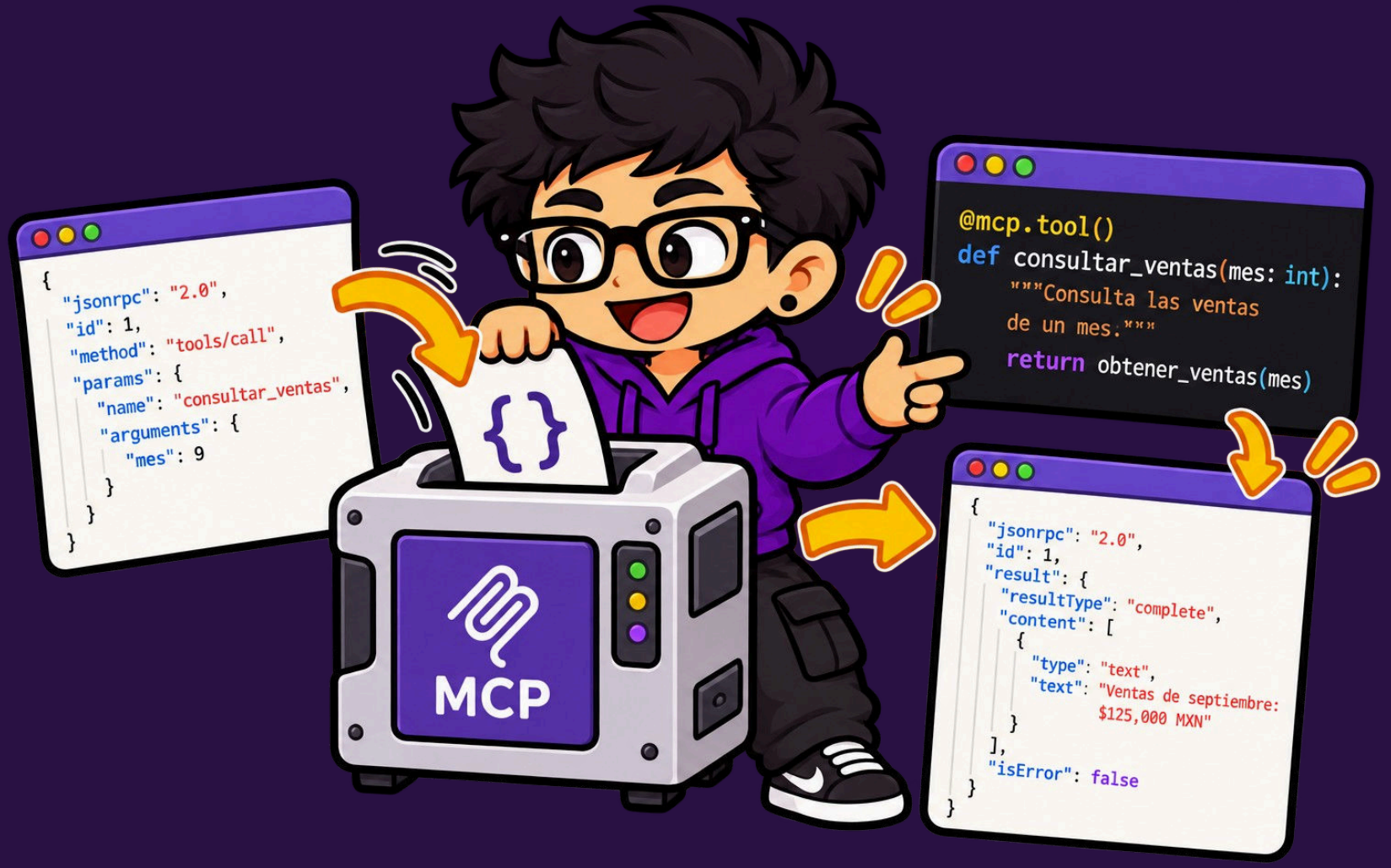
Una puerta.
Varios tipos de pedido.




```
https://api.empresa.com
│
├── /mcp
│   │
│   ├── server/discover
│   │   └── Capacidades del servidor
│   │
│   ├── tools/
│   │   ├── list
│   │   │   ├── consultar_ventas(mes)
│   │   │   ├── buscar_cliente(id)
│   │   │   └── generar_reporte(formato)
│   │   │
│   │   └── call
│   │       └── Ejecutar herramienta
│   │
│   ├── resources/
│   │   ├── list
│   │   │   ├── catalogo_productos
│   │   │   └── reporte_mensual
│   │   │
│   │   └── read
│   │       └── Consultar recurso
│   │
│   └── prompts/
│       ├── list
│       │   ├── analizar_ventas
│       │   └── generar_resumen
│       │
│       └── get
│           └── Obtener prompt
│
└── /.well-known/
    └── oauth-protected-resource
        └── Información de autorización
```

DEL MENSAJE A TU FUNCIÓN

Lo que mando ↔ lo que programo



Lo que mando ↔ lo que programo

```
{
  "name": "consultar_ventas",
  "description": "Consulta las ventas por mes",
  "inputSchema": {
    "type": "object",
    "properties": {
      "mes": {
        "type": "integer"
      }
    },
    "required": ["mes"]
  }
}
```

Lo que mando ↔ lo que programo

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "consultar_ventas",
    "arguments": {
      "mes": 9
    }
  }
}
```

Lo que mando ↔ lo que programo

```python
from mcp.server import MCPServer

mcp = MCPServer("Mi servidor")

@mcp.tool()
def consultar_ventas(mes: int) -> str:
    """Consulta las ventas de un mes."""
    return f"Ventas correspondientes al mes {mes}"

if __name__ == "__main__":
    mcp.run(transport="streamable-http")
```

Descubre. Decide. Ejecuta. Responde.

HOST · usuario + modelo + cliente MCP

SERVIDOR MCP

1	Cliente → servidor	`tools/list`
2	Servidor → host → modelo	**catálogo + esquemas**
3	Modelo propone → host autoriza	**consultar_ventas(mes=9)**
4	Cliente → servidor	`tools/call`
5	Servidor → backend → host	**resultado → respuesta al usuario**

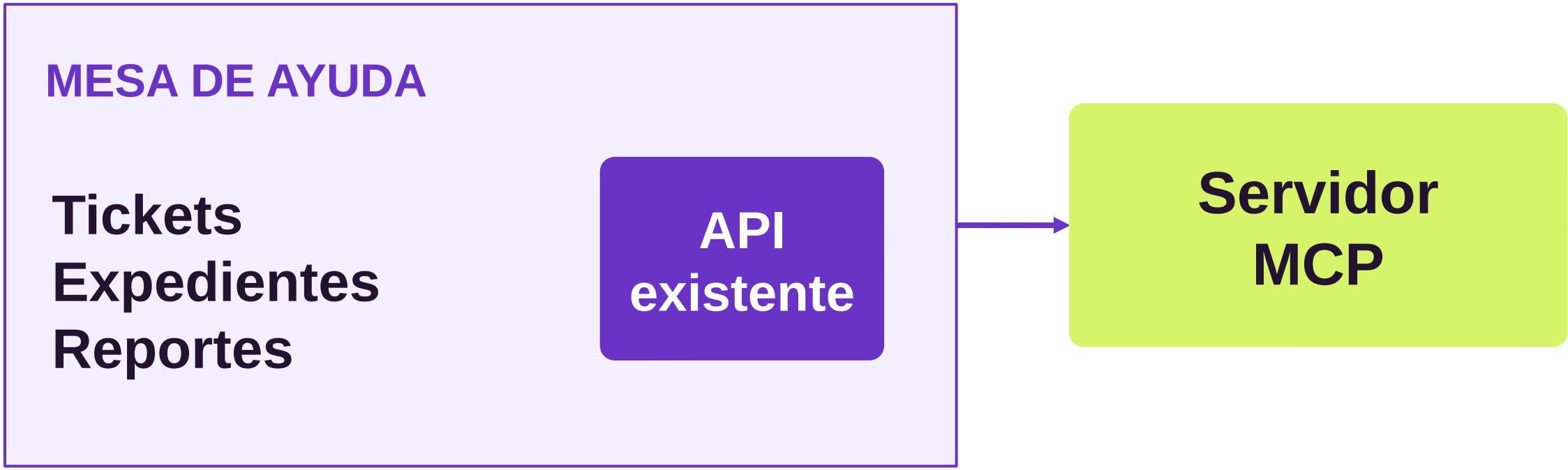
Tu herramienta conserva su lógica. MCP estandariza el acceso.

ACTO VI

Volvamos a la oficina

El sistema es de 2017.
La contraseña también.

Funciona desde 2017.



Las operaciones ya saben hacer el trabajo.

¿Un chatbot por cada tarea?

Chat de ventas

Chat de tickets

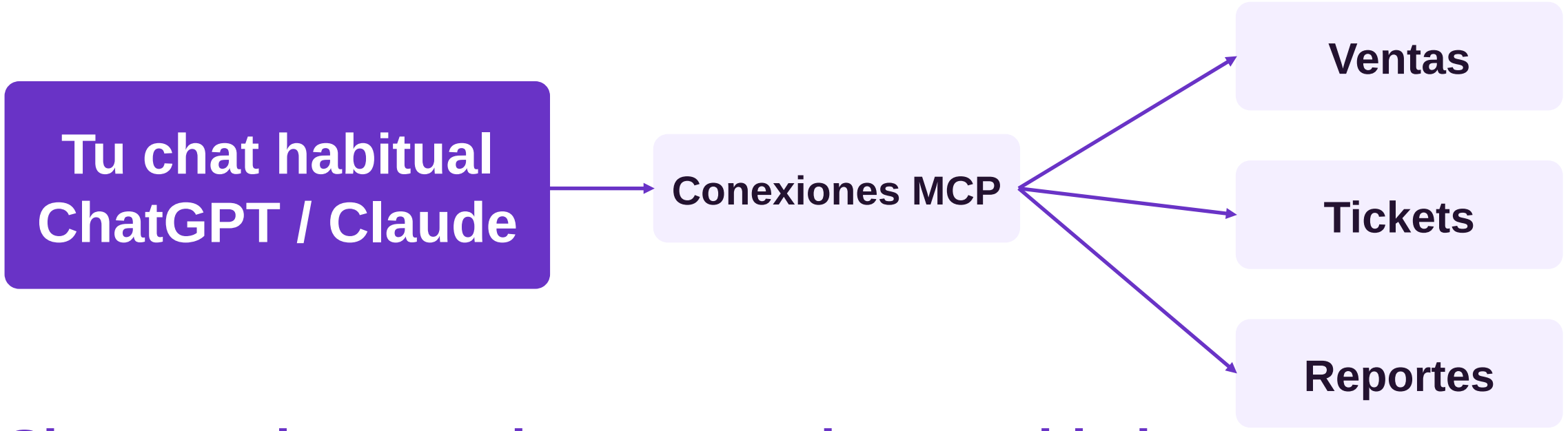
Chat de reportes



¡Otro chatbot corporativo™!

Ya tenemos un chat. Conectemos nuestras herramientas.

La misma IA. Más sistemas a su alcance.



Sin aprender otro chat para cada necesidad.

Soporte y configuración según producto, plan y permisos.
En desarrollo: Claude Code · Codex · OpenCode.

ACTO VII

Demos oficiales en ChatGPT

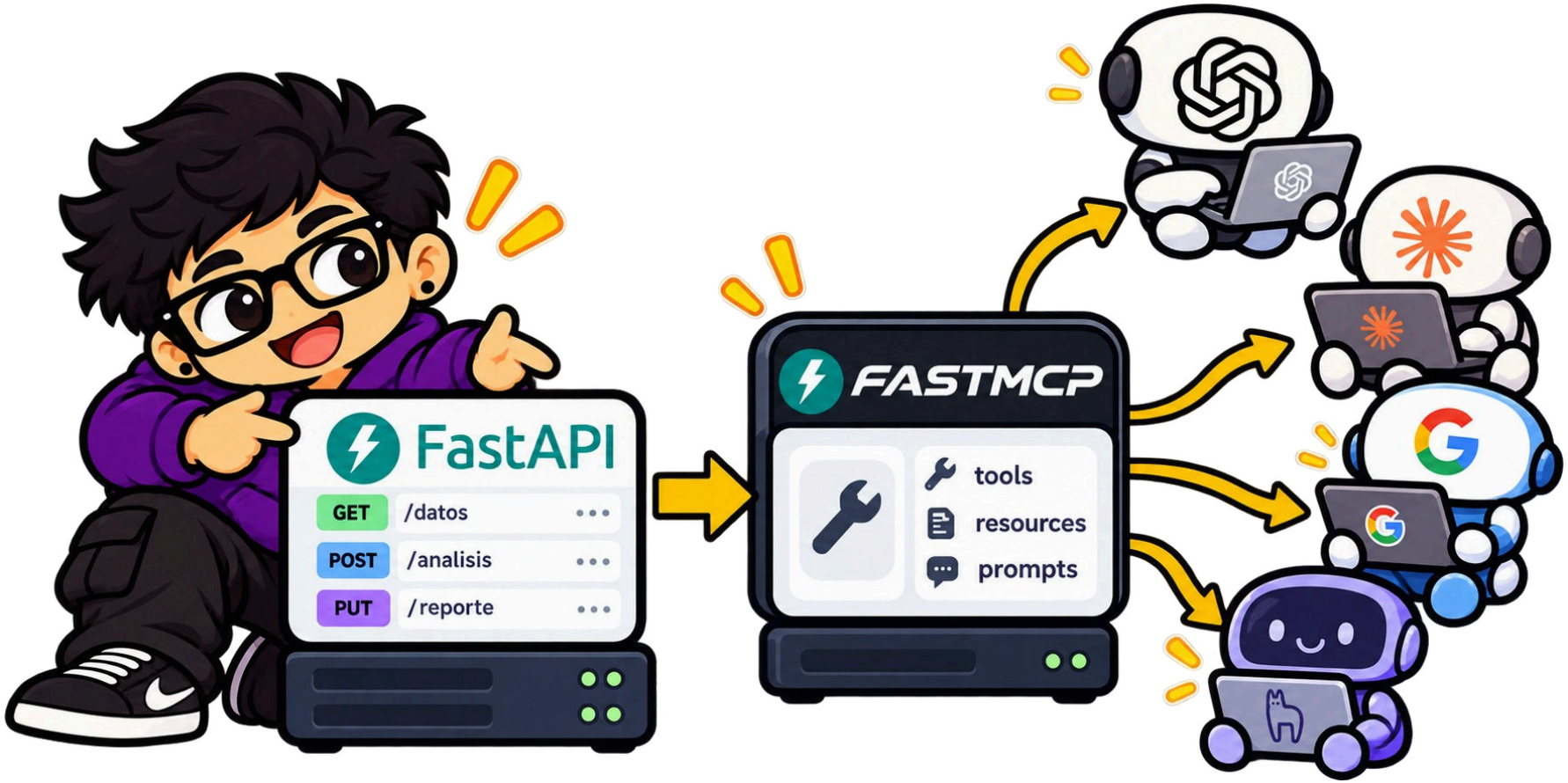
Primero: lo que ya existe.
Sin sacar la laptop a producción.

ACTO VIII

Ahora, mi propio sistema

Como todo sistema profesional… corre desde mi laptop.

De mi laptop a tu chat.



FastAPI → FastMCP → Tailscale Funnel → ChatGPT

Tu función. El contrato MCP.

MCP is Funny

```python
from fastmcp import FastMCP
import httpx

mcp = FastMCP("Mesa de ayuda")

@mcp.tool
def listar_tickets() -> list[dict]:
    """Consulta los tickets de la API."""
    r = httpx.get("http://127.0.0.1:8000/tickets")
    r.raise_for_status()
    return r.json()

mcp.run(transport="http", port=8001)

```

Convierte tu FastAPI a FastMCP

```python
from fastapi import FastAPI
from fastmcp import FastMCP

# Nuestra API existente
api = FastAPI()

@api.get("/ventas/{mes}", operation_id="consultar_ventas")
def consultar_ventas(mes: int) -> str:
    """Consulta las ventas de un mes."""
    return f"Ventas del mes {mes}"


# Convertimos nuestra API en un servidor MCP
mcp = FastMCP.from_fastapi(
    app=api,
    name="Mi servidor"
)

# Exponemos REST y MCP en la misma aplicación
mcp_app = mcp.http_app(path="/")

app = FastAPI(
    routes=api.routes,
    lifespan=mcp_app.lifespan
)

app.mount("/mcp", mcp_app)
```

ACTO VIII / DEMO

De mi laptop a tu chat.



FastAPI → FastMCP → Tailscale Funnel → ChatGPT

ACTO IX

Seguridad y cierre

Al alucín le dimos tools. ¿Qué podría salir mal?

Malas decisiones.
Ahora en más sistemas.



Autenticación	Autorización	Confirmación	Auditoría
¿Quién eres?	¿Qué puedes hacer?	¿Lo ejecutamos?	¿Qué ocurrió?

¿Y ahora quién sabe qué es MCP?



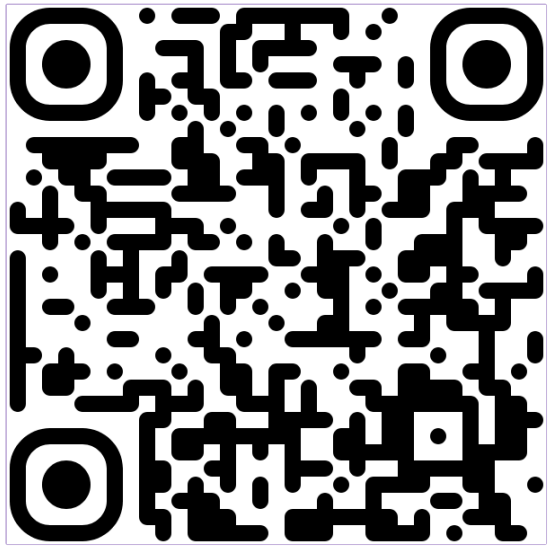
SIGAMOS LA CONVERSACIÓN

Nos vemos en las redes. Y en el repo.

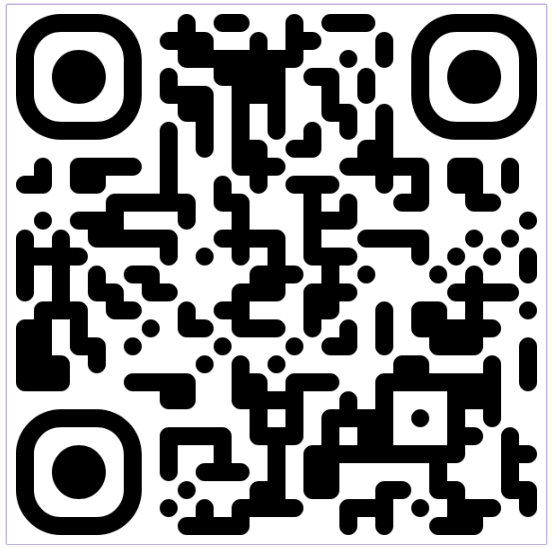
Linkedin



Github Repo



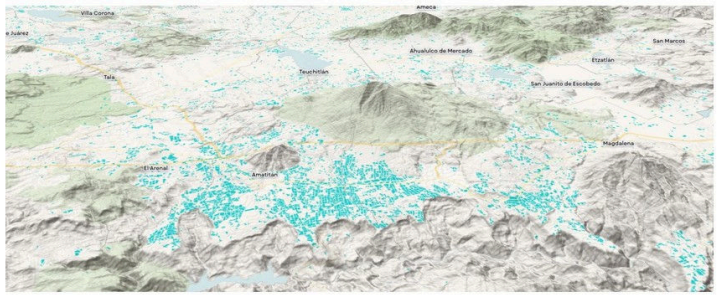
Datzin Blog



¡Síguenos en nuestro blog!



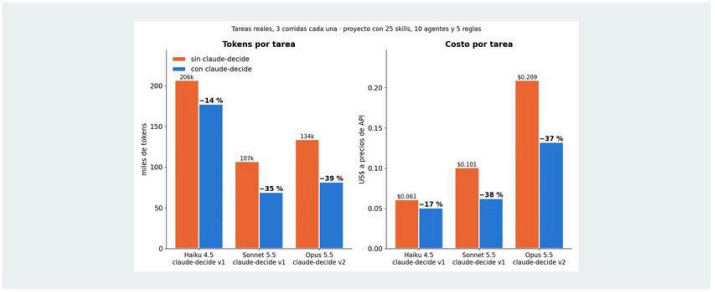
13 ARTÍCULOS



EDGAR VILLARREAL · PROYECTOS

MapaLab 2: menos plano, más interactivo

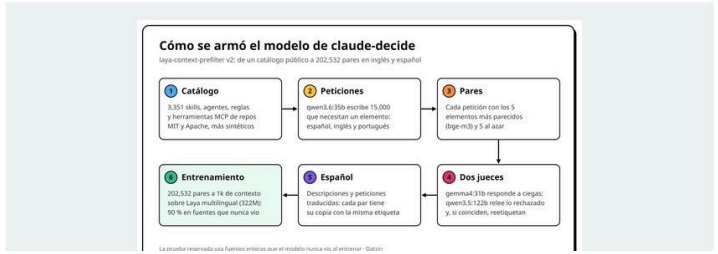
Para el análisis geoespacial: terreno en 3D, tablas, estadísticas y hexágonos en el visor del IIEG.

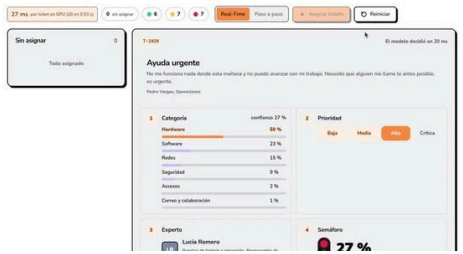


ALEJANDRO ZÁRATE MACÍAS · PROYECTOS

claude-decide: que Claude lea solo lo que necesita

Un modelo local de 322M decide qué skills, reglas, agentes y herramientas MCP
Claude Code en cada mensaje







¡Gracias por venir!

